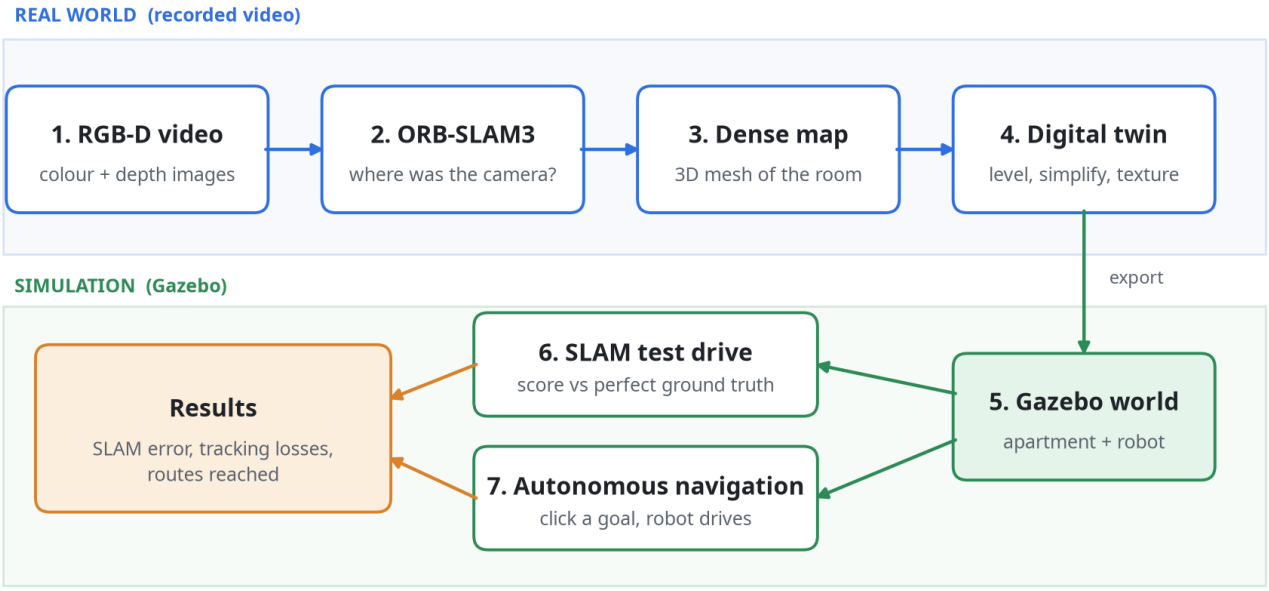
Visual SLAM to Autonomous Navigation

From real RGB-D video to a simulated apartment where a robot maps, is tested against perfect ground truth, and drives itself.



The whole project on one page: real video goes in at the top, a working robot simulation comes out at the bottom.

Project guide and study notes · Azad Kumar Jha · 13 October 2026
Code: ~/github/visual_slam · Tools: ORB-SLAM3, Open3D, Gazebo Harmonic, Python

Contents

1. The big picture 3

2. The basics you need 5

3. ORB-SLAM3: the SLAM engine 9

4. First maps from real video 11

5. From points to a solid 3D model 14

6. Building the digital twin 15

7. Gazebo: the robot simulator 17

8. Testing SLAM inside the twin 19

9. Making the robot drive itself 23

10. Problems we hit and how we fixed them 26

11. How to run everything 28

12. What is next 29

Glossary 30

How to read this guide

Each chapter starts with an **In simple words** box like this one. Read only those boxes for a 10-minute overview; read the rest when you want the details. Orange **Lesson learned** boxes mark mistakes we made and how we caught them. A glossary of every technical word is at the end.

1. The big picture

> **In simple words**
> We took videos of real rooms, used them to build an accurate 3D copy of an apartment, put that copy into a robot simulator, and then (a) tested how well a robot can map the room from its own camera, and (b) made the robot drive itself to any point you click.

The project has seven stages. Each one produces something the next stage uses:

1. **Record**: colour + depth video from public datasets (TUM and Stanford BundleFusion).
2. **Localize**: ORB-SLAM3 works out where the camera was for every frame.
3. **Map**: depth images are fused at those positions into a 3D mesh of the room.
4. **Twin**: the mesh is levelled, simplified and coloured, then turned into a Gazebo world (a “digital twin” of the real apartment).
5. **Robot**: a small wheeled robot with a lidar, a camera and an IMU is placed inside.
6. **Test SLAM**: the robot drives a loop; ORB-SLAM3 maps from its camera and is scored against the simulator's exact position.
7. **Navigate**: click a point on a floor plan; the robot plans a route and drives there, avoiding surprise obstacles.

Headline results

What	Result
ORB-SLAM3 on a real office video (22 m loop)	1.0 cm average position error
ORB-SLAM3 on the real apartment scan (89 m)	One continuous map; 9.7 cm vs reference
3D apartment model	5.9 x 5.8 m, 1 cm detail, 1 million triangles, sharp colour texture
SLAM from the robot's camera in the simulator	2.1 cm error over 15 m, never lost track
Autonomous navigation	Reaches clicked goals within about 10 cm; drives around a surprise box, or reports when the way is blocked



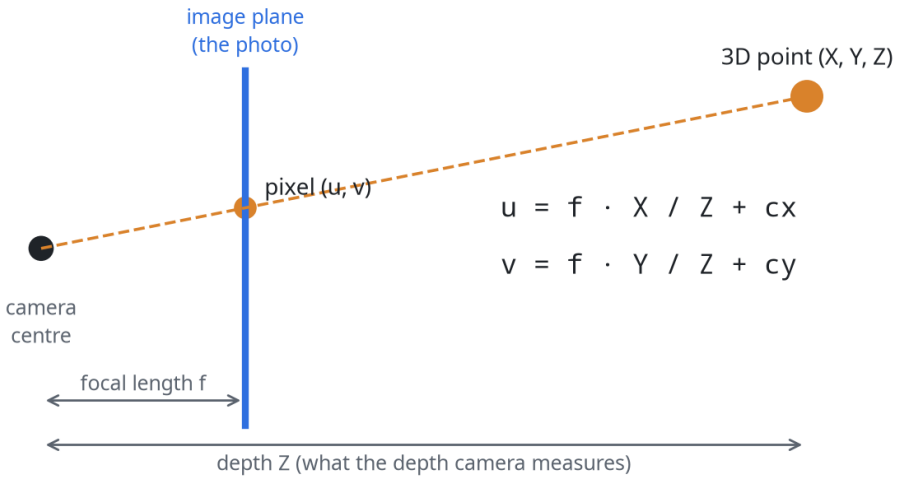
The final result: our robot (blue) inside the 3D copy of the real apartment, rendered by Gazebo. The bookshelf, curtain, table and chairs all come from the scan.

2. The basics you need

> **In simple words**
> A camera turns 3D points into 2D pixels. SLAM runs this backwards: from many photos it figures out both *where the camera was* and *where things are*. It does this by spotting the same small details (features) in many photos and finding the positions that explain them best.

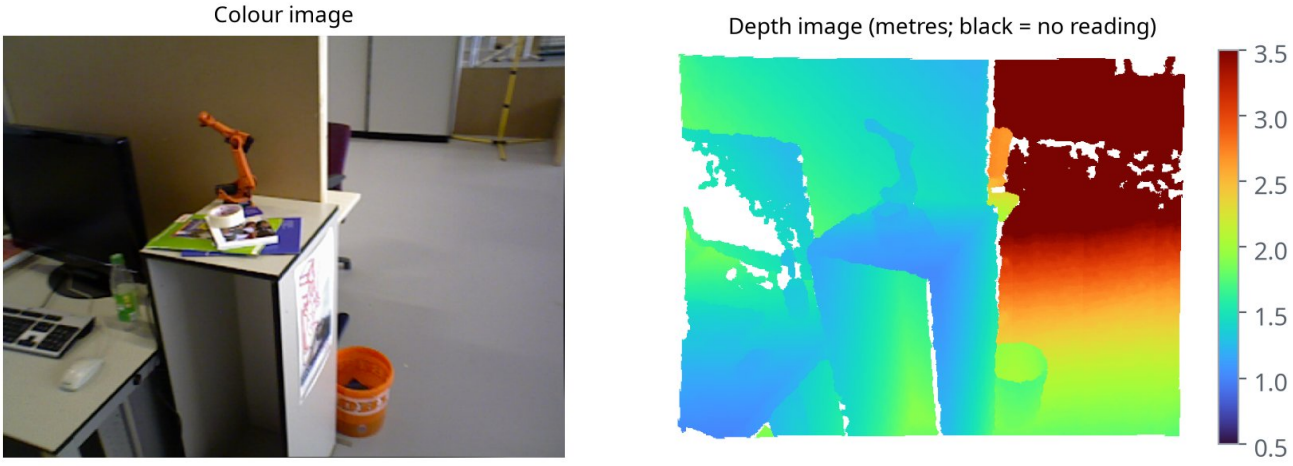
2.1 How a camera sees

A camera is like a pinhole: light from a 3D point travels in a straight line through the camera centre and lands on the image. The farther away the point (bigger Z), the closer to the centre of the image it lands. Four numbers describe a camera: two focal lengths (fx, fy, how zoomed-in it is) and the image centre (cx, cy). Together they are called the **intrinsics**.



The pinhole camera model: a 3D point is projected through the camera centre onto the image plane.

A normal camera loses depth: from one photo you cannot tell a small near object from a big far one. An **RGB-D camera** (like the Kinect or Structure Sensor) also measures depth for every pixel. That is why every main run in this project used RGB-D: the depth gives true sizes in metres.

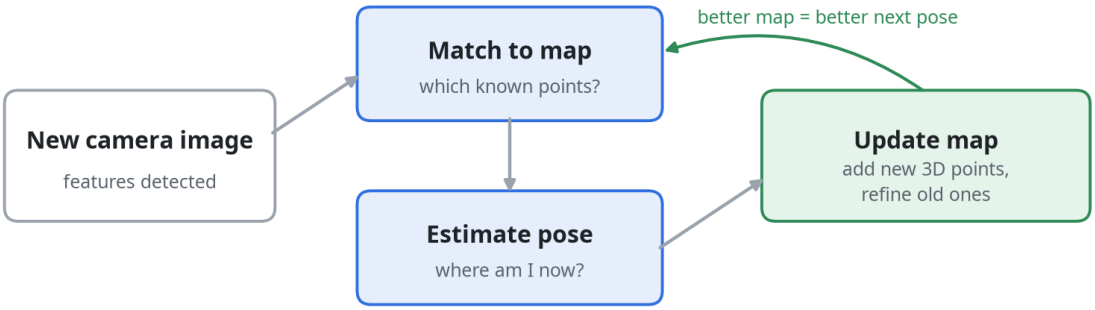


One RGB-D frame from the TUM office dataset: the colour image and its depth image. Each depth pixel is a distance in metres (stored as a 16-bit number; divide by 5000).

Camera	fx	fy	cx	cy	Depth unit
TUM fr3 (Kinect)	535.4	539.2	320.1	247.6	1/5000 m
BundleFusion (Structure Sensor)	582.9	582.9	320	240	1/1000 m
Gazebo robot camera (60° view)	554.4	554.4	320	240	metres (float)

2.2 What SLAM does

SLAM stands for **Simultaneous Localization And Mapping**. Localization needs a map, and building a map needs to know where you are, so SLAM does both at once in a loop:



The SLAM loop: each new image is matched to the map, the pose is estimated, and the map is improved, which makes the next pose easier.

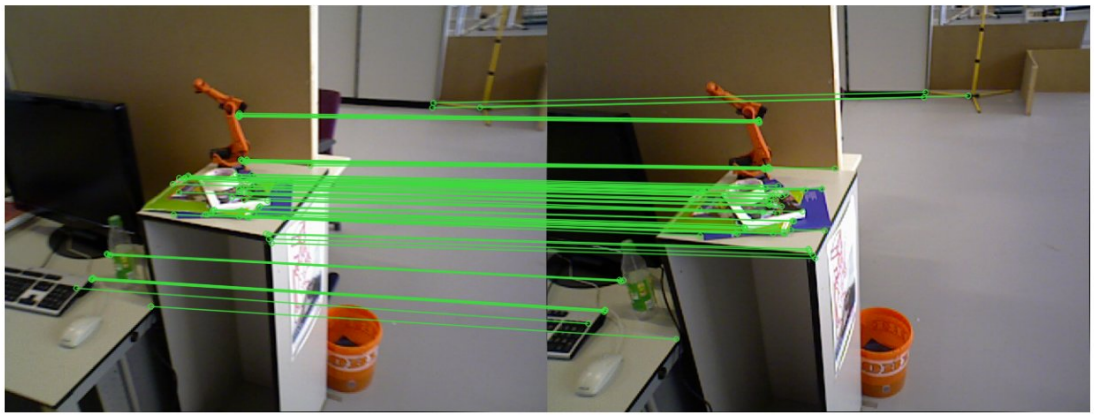
2.3 Features: the details SLAM tracks

SLAM does not use every pixel. It picks **features**: small, distinctive spots such as corners and edges of objects. ORB-SLAM3 uses **ORB features**. Each one gets a 256-bit “fingerprint” (descriptor), so the same spot can be recognised in another photo by comparing fingerprints.

800 ORB features (yellow dots): they gather on textured objects, not on blank walls



Matching the same features in a frame half a second later (70 best matches)



Top: ORB features found in a real frame. Bottom: the same features matched in a frame taken half a second later. Blank walls give almost no features.

> **Why this matters later**
> Features need texture: patterns, edges, writing, objects. A smooth white wall or a blurry surface has almost no features, and SLAM gets lost there. Remember this; it explains most of the simulation results in chapter 8.

2.4 Pose: position + rotation

A camera's **pose** is where it is (x, y, z) and which way it is facing (3 rotation angles): 6 numbers in total. It is stored as a 4x4 matrix. Rotations are tricky to optimize because you cannot simply add numbers to a rotation and still have a valid rotation. SLAM software therefore works with a small 6-number change (“perturbation”) and applies it with a special function (the exponential map, from Lie group theory). You do not need the maths to use SLAM, but it is the first lab of the from-scratch course in this project (labs/00_lie_groups.md).

2.5 Finding the pose from known points (PnP)

If the map already contains 3D points and we can see some of them in a new photo, we can compute where the camera must be. This is called **Perspective-n-Point (PnP)**. Some matches are always wrong, so **RANSAC** is used: try many small random sets of matches, keep the answer most matches agree with, and throw away the rest as outliers.

2.6 Bundle adjustment: polishing everything together

Small errors build up. **Bundle adjustment (BA)** fixes this by adjusting all camera poses and all 3D points together so that every point projects as close as possible to where it was actually seen. The “error” it minimizes is the distance in pixels between where a point should appear and where it was seen (the **reprojection error**).

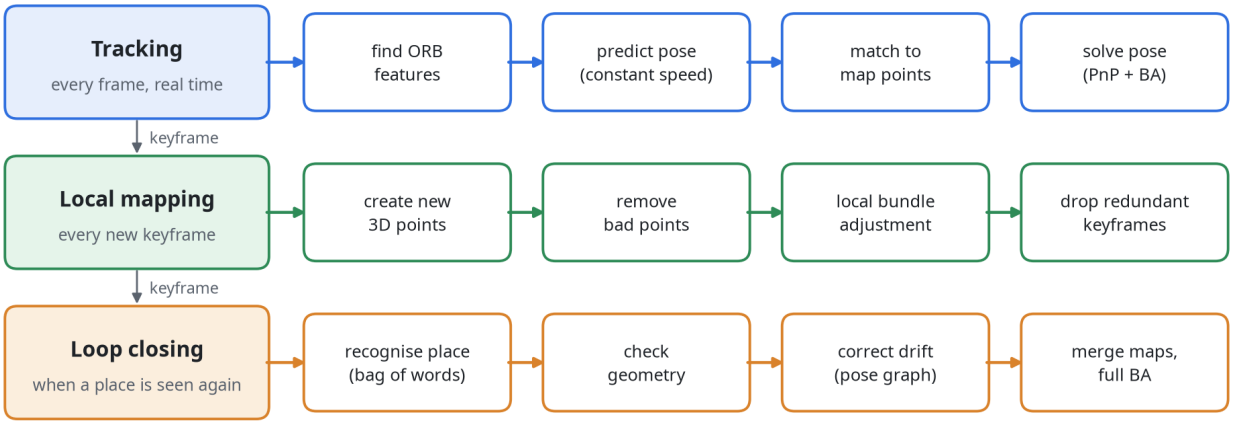
$$\texttt{minimize: sum of (projected point - observed pixel)}^2$$

2.7 Scale: why depth matters

With a single normal camera, a scene and a copy of it twice as big look identical, so the map's **scale** is unknown and slowly drifts. In an early test the map shrank to 65% of its true size. An RGB-D camera measures real distance, so its maps have the correct size.

3. ORB-SLAM3: the SLAM engine

In simple words
ORB-SLAM3 is a well-known, open-source SLAM system. It runs three jobs at the same time: one follows the camera in every frame, one builds and polishes the map, and one recognises places it has seen before to remove accumulated errors.



ORB-SLAM3's three threads and what each one does.

Key ideas

- **Keyframes**: only some frames are kept in the map (when the view has changed enough). This keeps the map small and fast.
- **Covisibility graph**: keyframes that see the same points are linked, so ORB-SLAM3 only has to work with the nearby part of the map.
- **Bag of words**: features are grouped into a “visual vocabulary” (about a million words, file ORBvoc.txt). Each keyframe becomes a word list, so similar places can be found instantly, like a search engine for images.
- **Loop closure**: when the camera returns to a known place, the drift that built up along the way is spread out and corrected over the whole path.
- **Atlas and map merging**: if tracking is lost, ORB-SLAM3 starts a new map. If it later recognises a place from an old map, it merges the two (“Merge detected”).

Getting it to build

ORB-SLAM3 is from 2022 and did not compile on Ubuntu 24.04 out of the box. We installed the build tools, then fixed four errors: Pangolin (its 3D viewer) needed an extra header, both had to use the C++17 standard, one variable that was incremented had to change from bool to int, and we added our own function to save the map points (ORB-SLAM3 only saves trajectories). ORB-SLAM3 also tends to crash when its viewer closes, after it has saved everything, so our run script checks for the saved trajectory instead of trusting the exit code.

What it outputs

File	What is inside
CameraTrajectory.txt	The pose of every frame: time, position, rotation (quaternion)
KeyFrameTrajectory.txt	The poses of the keyframes only
MapPoints.txt	The sparse 3D map (added by us): map id, x, y, z, number of observations

4. First maps from real video

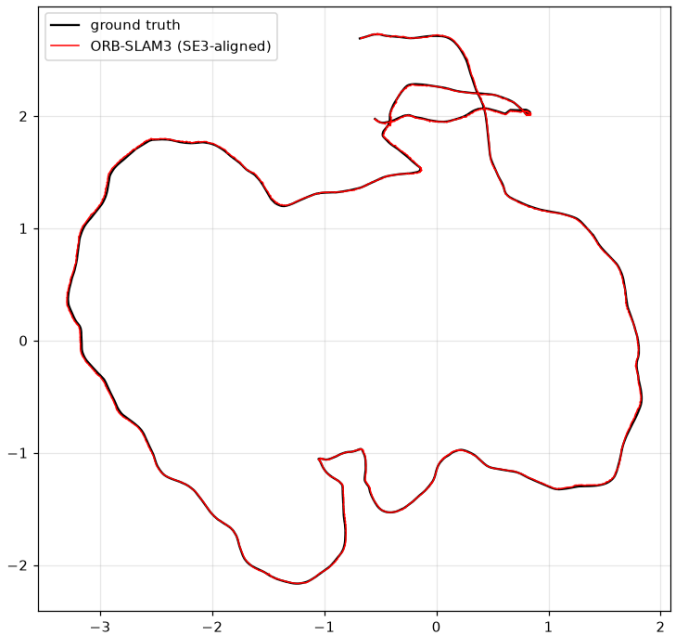
> **In simple words**
> We ran ORB-SLAM3 on two real recordings: an office (with a motion-capture system that knows the true camera path) and a whole apartment. In the office, ORB-SLAM3's path was within 1 cm of the truth on average over 22 metres.

4.1 How we measure accuracy

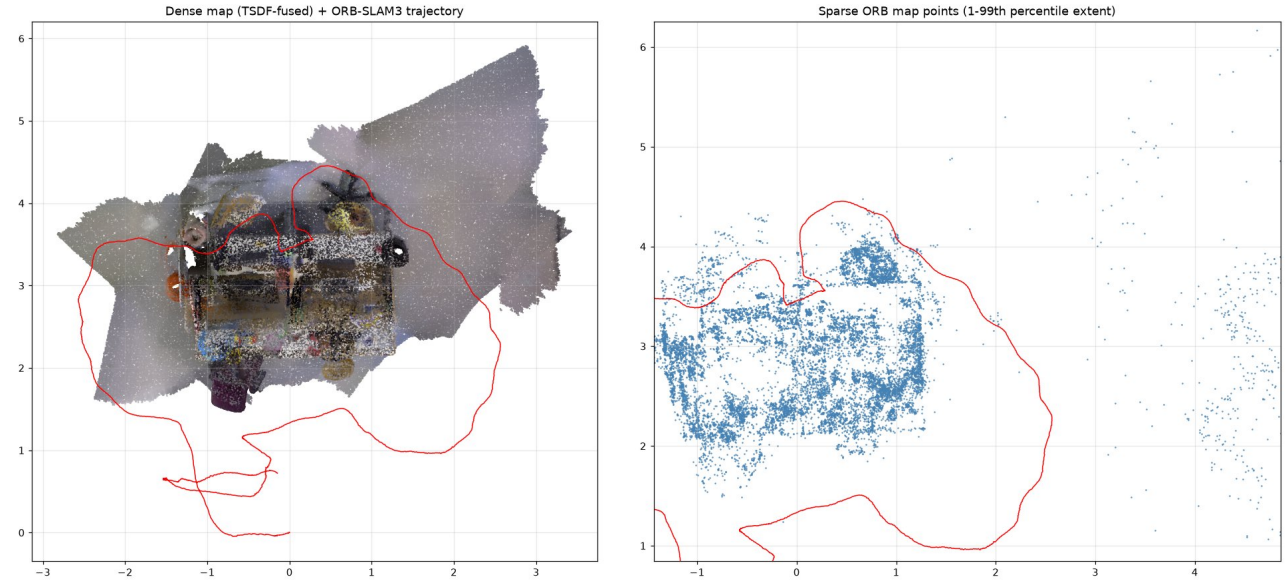
The SLAM path and the true path are in different coordinate systems, so we first line them up (**Umeyama alignment**: the best rotation + shift, and for single-camera runs also a scale). Then **ATE** (Absolute Trajectory Error) is the average distance between matching points of the two paths (root-mean-square). Smaller is better.

4.2 The office: TUM fr3_long_office_household

2,488 RGB-D frames, 2.5 minutes, the camera circles two desks and returns to the start. ORB-SLAM3 recognised the start and **closed the loop**.



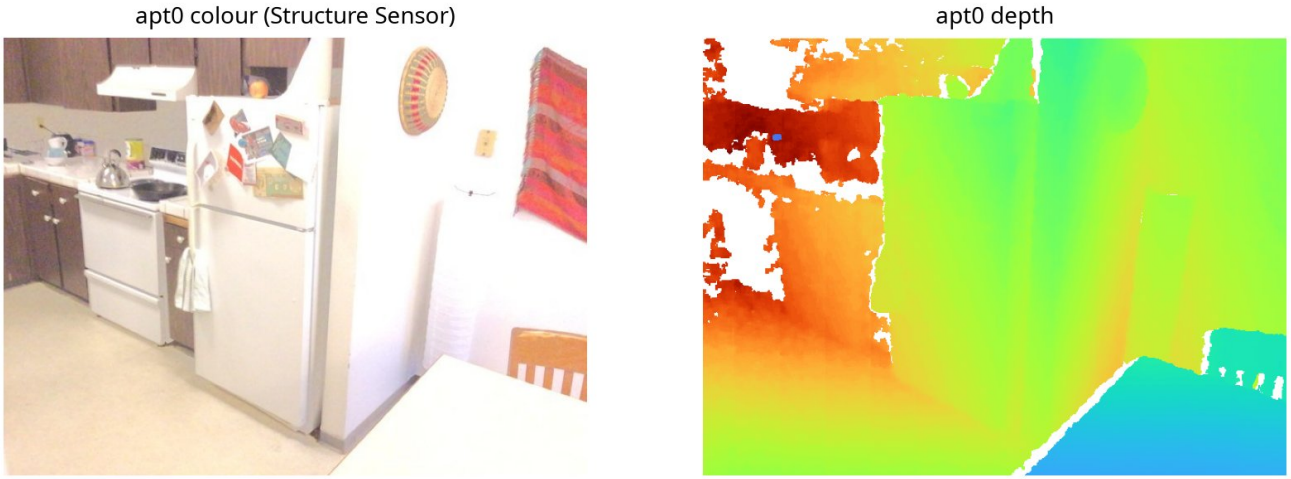
Black: the true camera path (motion capture). Red: ORB-SLAM3's estimate. They overlap almost perfectly: 1.01 cm average error, 3 cm worst case, over 22.1 m.



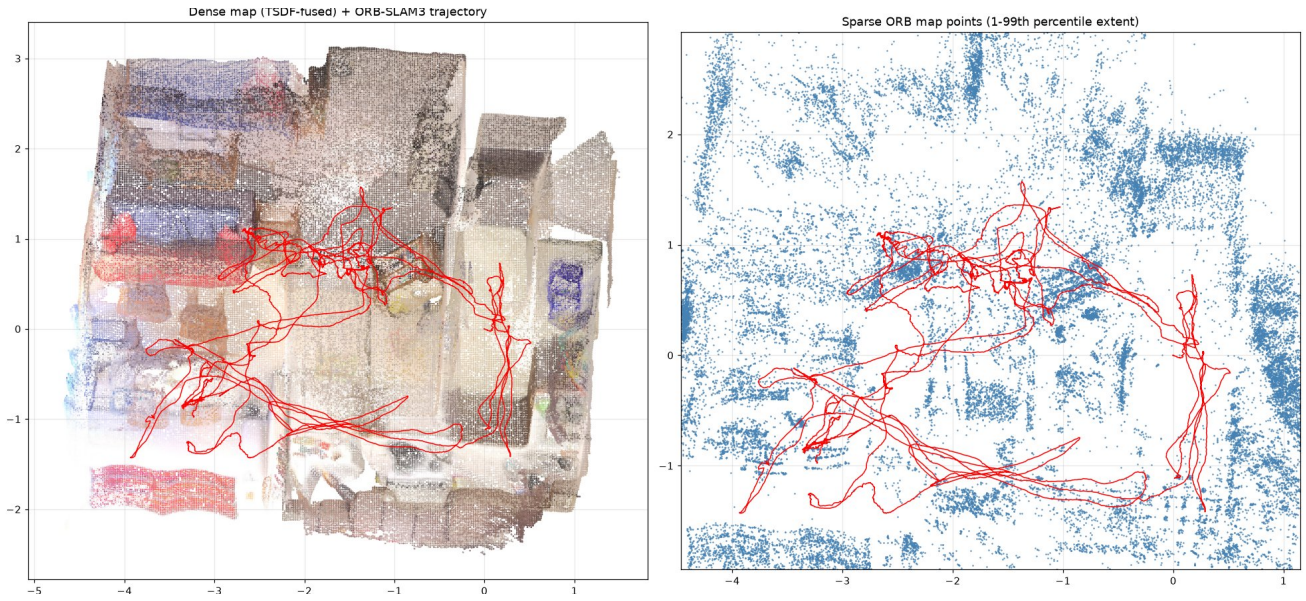
Top view of the office map. Left: the dense coloured map with the camera path. Right: ORB-SLAM3's sparse feature points.

4.3 The apartment: Stanford BundleFusion apt0

8,560 frames from a Structure Sensor carried through a multi-room apartment, an 89 m walk. There is no motion capture here; the reference path comes from another system (BundleFusion), so the 9.7 cm difference is agreement, not true error. ORB-SLAM3 kept one continuous map but did not detect a loop, so drift was not corrected.



One frame of the apartment scan: colour and depth.



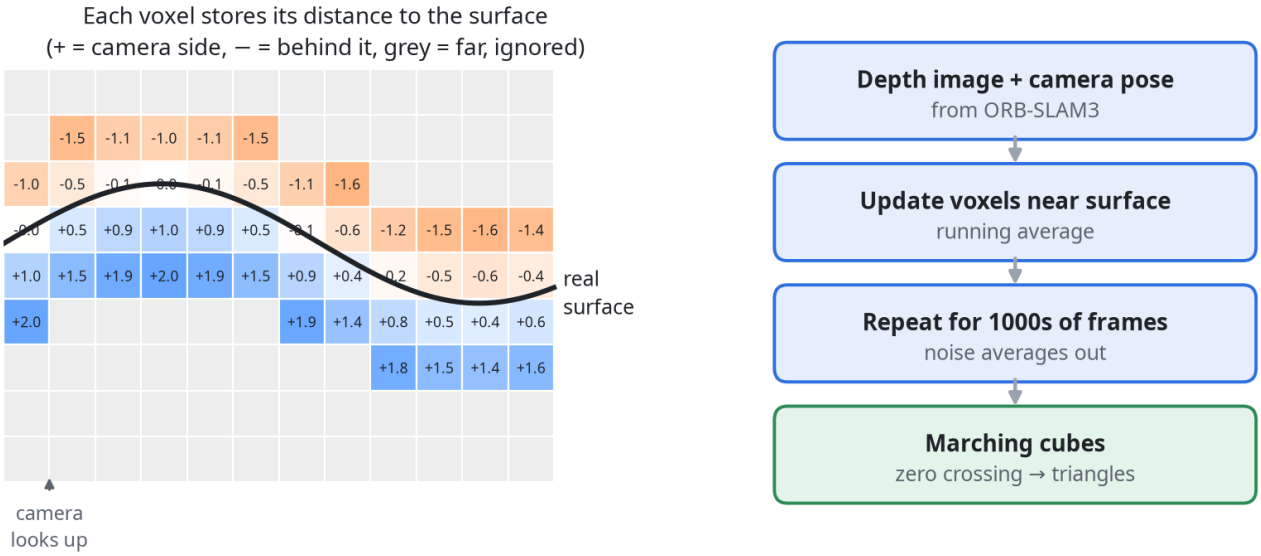
Top view of the apartment: rooms, sofa and furniture, with the walking path in red.

Run	Camera	Error	Notes
Our own early prototype, fr1_xyz	Single	8.2 cm	Map shrank to 65% scale
ORB-SLAM3, office (fr3)	RGB-D	1.0 cm	Loop closed; 34 ms per frame
ORB-SLAM3, apartment (apt0)	RGB-D	9.7 cm vs reference	One map; 16 ms per frame

5. From points to a solid 3D model

In simple words
ORB-SLAM3's map is just a cloud of feature points, good for navigation but not something you can look at. To get solid walls and furniture, we take every depth image, place it where ORB-SLAM3 says the camera was, and blend them all into one smooth surface.

The method is called **TSDF fusion** (Truncated Signed Distance Function). Space is divided into tiny cubes (**voxels**, 1 or 2 cm). Each voxel remembers how far it is from the nearest surface: positive on the camera's side, negative behind the surface. Every depth image updates these numbers with a running average, so sensor noise averages away. The surface is exactly where the numbers change from + to -. **Marching cubes** then turns that boundary into triangles.



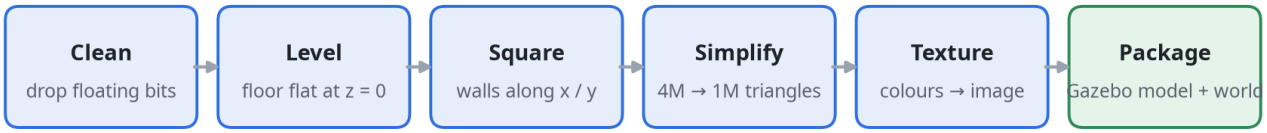
Left: a 2D slice of a TSDF. The black line is the real surface; voxel numbers are distances to it. Right: the fusion steps.

Map	Voxel size	Frames used	Triangles	Time
Office (fr3)	1 cm	830	1.1 million	18 s
Apartment (apt0)	2 cm	2,854	0.9 million	about 2 min
Apartment HD	1 cm	2,854	4.2 million	60 s

Smaller voxels give more detail but four times more triangles each time the size halves. Chapter 8 shows that this detail turned out to matter a lot.

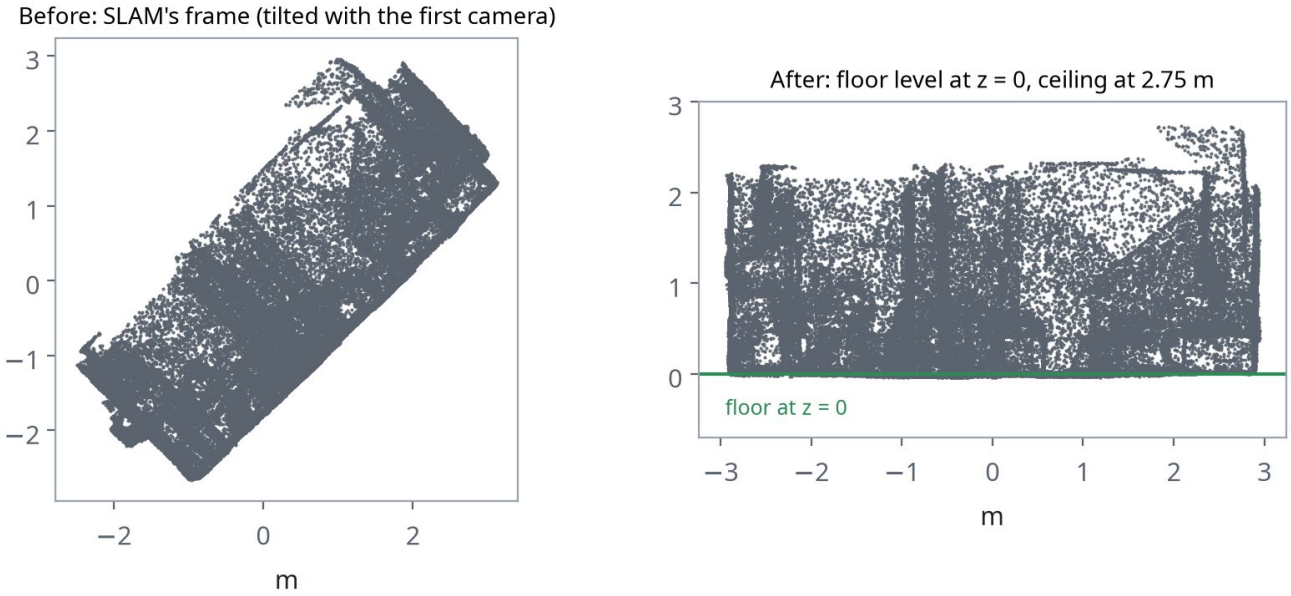
6. Building the digital twin

In simple words
The raw 3D model is tilted (it uses the first camera's direction as “up”), too heavy for a simulator, and coloured in a way Gazebo cannot show. Six steps turn it into a clean, level, light Gazebo model of the apartment.



The six export steps in tools/export_gazebo.py.

- **Clean**: delete small floating pieces (948 of 953 pieces were noise).
- **Level**: find the floor with **RANSAC plane fitting** and rotate it flat. “Down” is estimated as the camera's down direction averaged over the whole walk (people hold a scanner roughly upright on average). The floor is the lowest large horizontal plane, so a big table is not mistaken for it.
- **Square**: rotate so the walls line up with the x and y axes, and centre the apartment.
- **Simplify**: reduce triangles (quadric decimation) while keeping the shape. One mesh for looks (1 million triangles) and a lighter one for physics (0.1 million).
- **Texture**: Gazebo shows colour from an image, not from points, so colours are “baked” into an 8192 x 8192 image.
- **Package**: write the Gazebo model files and a world file.



Side view of the apartment before and after levelling. After: floor at 0 m, ceiling at 2.75 m, and the scanner path 1.45 m above the floor (chest height), a good sanity check.

Lesson learned

The first levelling attempt put the whole apartment 1.4 m *underground* because of a sign error in one formula. It was caught because the camera height came out negative (-1.07 m). Always compare outputs with physical common sense: room height, person height, floor at zero.

Texture without the headache

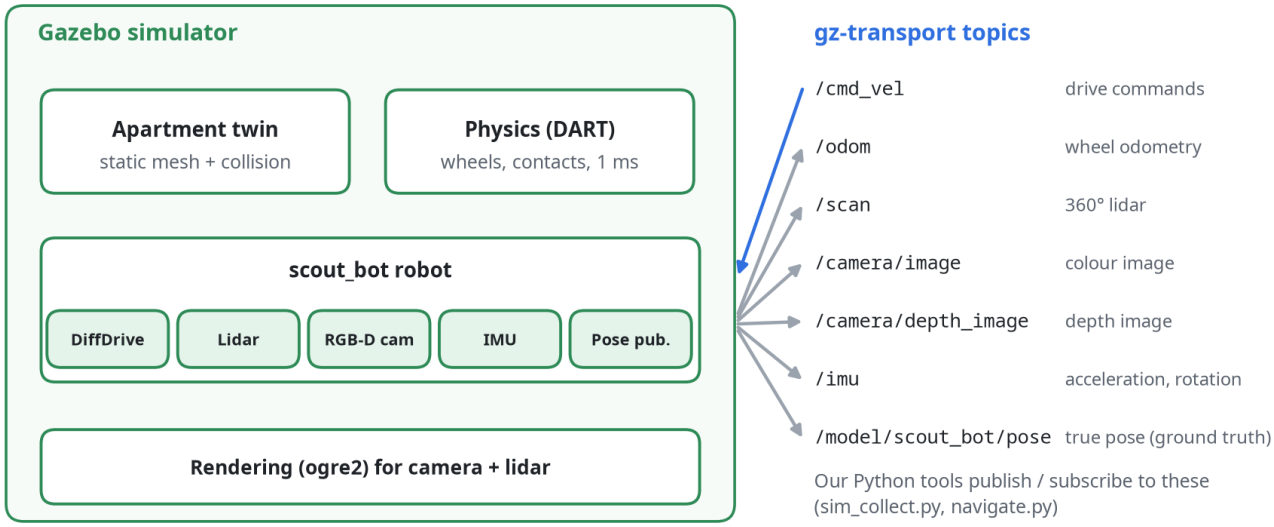
The standard texturing tool (UVAtlas) needs a perfectly clean mesh and took more than 10 minutes on 1 million triangles. We wrote a faster method: give every triangle its own small square in the image and colour it from the full-detail 4-million-triangle model. Triangles that are neighbours in the room are placed next to each other in the image (a **Morton / Z-order** curve), so blending at the edges mixes similar colours and no seams appear.



The finished twin in Gazebo, seen from just under the ceiling. The robot is the small blue square in the middle.

7. Gazebo: the robot simulator

In simple words
Gazebo is a free robot simulator. It handles physics (wheels, collisions), draws what cameras and lidars would see, and lets programs talk to the robot through named message channels called **topics**. We used Gazebo Harmonic, the current long-term release.

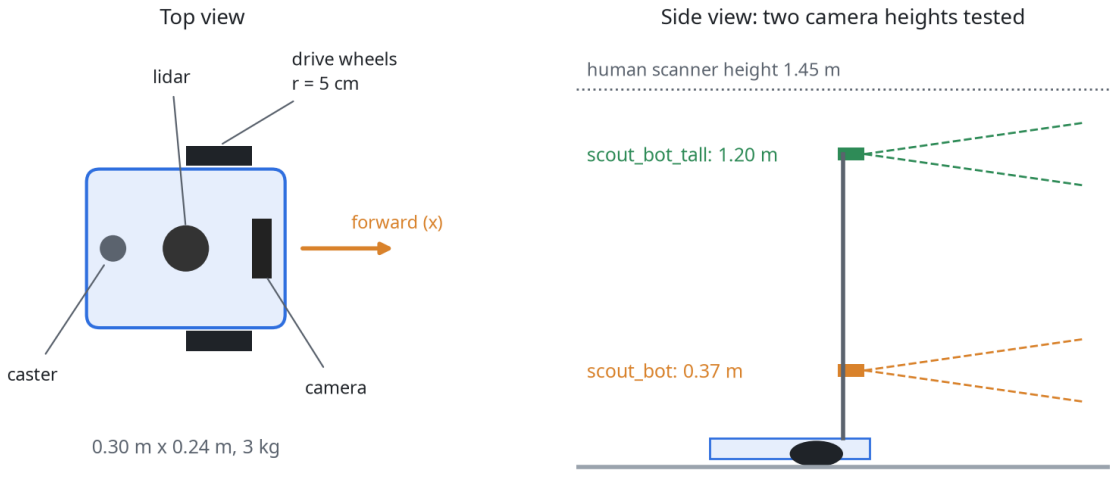


What runs inside Gazebo, and the topics our Python tools use to read sensors and send drive commands.

7.1 Worlds and models (SDF files)

Everything in Gazebo is described in **SDF**, a simple XML format. A **model** is a folder with model.sdf (shapes, joints, sensors) and model.config (name). A **world** file lists the physics settings, the light, the GUI layout and which models to include. The apartment is a *static* model: it never moves. It has a visual mesh (what you see) and a collision mesh (what the robot bumps into).

7.2 The robot: scout_bot



Left: the robot from above. Right: the two camera heights we tested.

Part	Gazebo element	Topic	Details
Wheels	DiffDrive plugin	/cmd_vel (in), /odom (out)	2 driven wheels, r = 5 cm
Lidar	gpu_lidar sensor	/scan	360 beams, 10 per second, up to 8 m
Camera	rgbd_camera sensor	/camera/image, /camera/depth_image	640 x 480, 15 per second
IMU	imu sensor	/imu	100 per second
True position	PosePublisher plugin	/model/scout_bot/pose	Exact pose, 100 per second

7.3 Talking to Gazebo from Python

Gazebo's message system (**gz-transport**) has Python bindings. Our scripts subscribe to topics (camera images, lidar scans, the true pose) and publish to /cmd_vel to drive. Services are used to add and remove objects while running (we dropped test boxes this way) and to draw the planned route in the 3D view (the /marker service, provided by the MarkerManager GUI plugin).

```
gz topic -l                                  # list all topics
gz topic -e -t /scan -n 1                    # print one lidar message
gz topic -t /cmd_vel -m gz.msgs.Twist -p 'linear: {x: 0.2}'   # drive forward
gz sim -s -r --headless-rendering world.sdf   # run without a window (used for all tests)
```

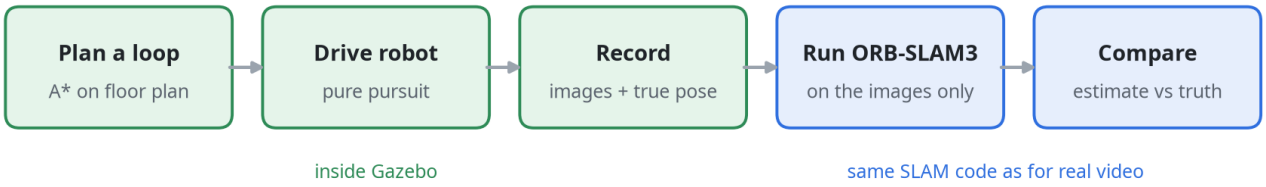
7.4 GUI panels used

3D view, play/pause, world stats, the robot camera image (ImageDisplay), lidar rays (VisualizeLidar), keyboard driving (KeyPublisher with the arrow keys), and route drawing (MarkerManager). Right-click the robot and choose **Follow** to keep it in view.

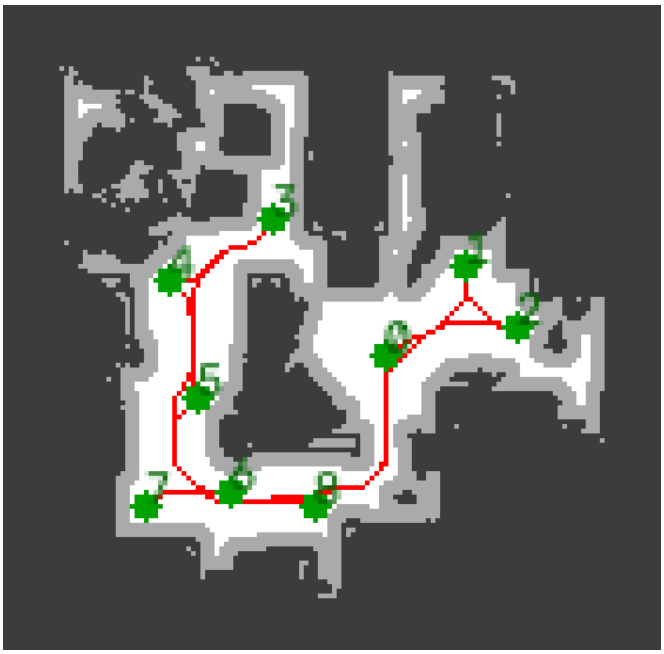
8. Testing SLAM inside the twin

In simple words

In real life you never know exactly where a camera was. In the simulator you do. So we drove the robot around the apartment, fed its camera images to ORB-SLAM3, and compared ORB-SLAM3's answer with the exact truth. Then we changed one thing at a time to learn what makes SLAM succeed or fail.



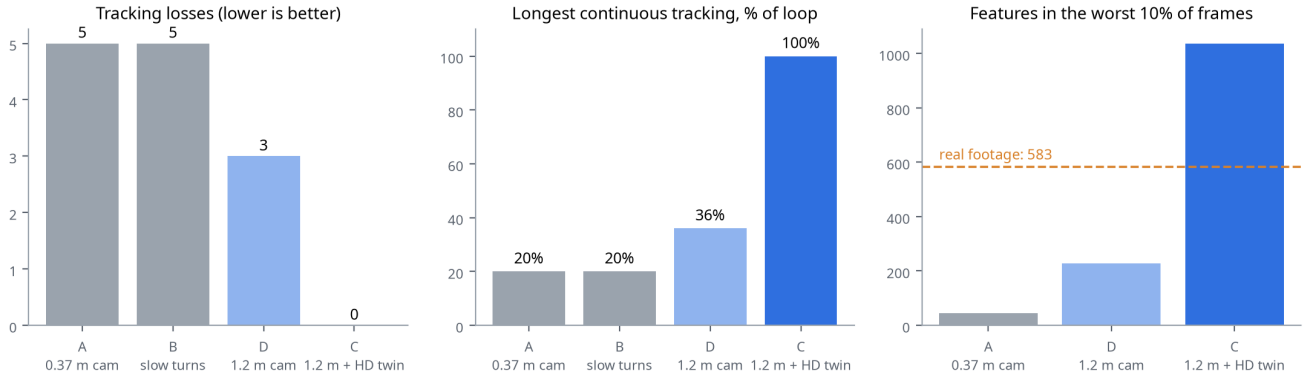
One test run, start to finish (tools/sim_collect.py, run_orbslam3.sh, eval_segments.py).



The planned test loop: 8 spread-out waypoints, visited in order and back to the start, on the floor plan made from the scan.

8.1 The four experiments

Run	What changed	Lost track	Longest stretch tracked	Error while tracking
A	Starting point: camera at 0.37 m, 2 cm model	5 times	3.4 m (20%)	about 1 cm
B	Slower turns	5 times, same places	3.5 m (20%)	about 1 cm
D	Camera raised to 1.2 m	3 times (2 maps merged)	5.5 m (36%)	about 1 cm
C	1.2 m camera + 1 cm HD model	never	14.9 m (100%)	2.1 cm over the whole loop



Left to right: how often tracking was lost, how much of the loop was tracked without a break, and how many features the hardest 10% of frames had (orange: the real footage).

Run A: camera at 0.37 m, 2 cm twin
(holes, undersides never scanned)



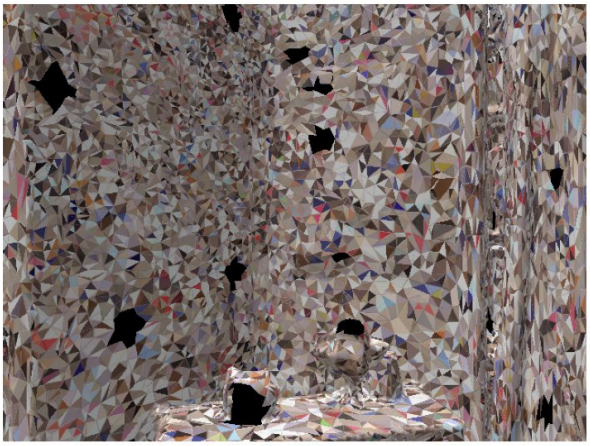
Run D: camera at 1.2 m, 2 cm twin
(sees what the scanner saw; blurry)



Run C: camera at 1.2 m, 1 cm HD twin
(sharp: poster text readable)



INVALID: texture bug ("confetti")
fake detail fooled the score



What the robot's camera saw in each run. The low camera (A) sees holes and undersides the human scanner never filmed. The HD model (C) is sharp enough to read the poster.

8.2 What each run taught us

- **A**: when ORB-SLAM3 was tracking, it was accurate (about 1 cm). The problem was that it kept losing track.
- **B**: slowing down did nothing; it lost track at the very same spots. So the cause was the *scene*, not the motion.
- **D**: from 37 cm high, the camera saw surfaces the scanner (carried at 1.45 m) never filmed: holes, ragged edges, grey gaps. At 1.2 m it saw what the scanner saw, features doubled, and ORB-SLAM3 could even recognise places again.
- **C**: the last problem was blurry texture. With the sharper 1 cm model, tracking never broke.

The main finding
A scanned copy of a room is only trustworthy for testing a robot if it was scanned from the robot's own viewpoint and in enough detail. Fix both, and simulated SLAM behaves like real SLAM.

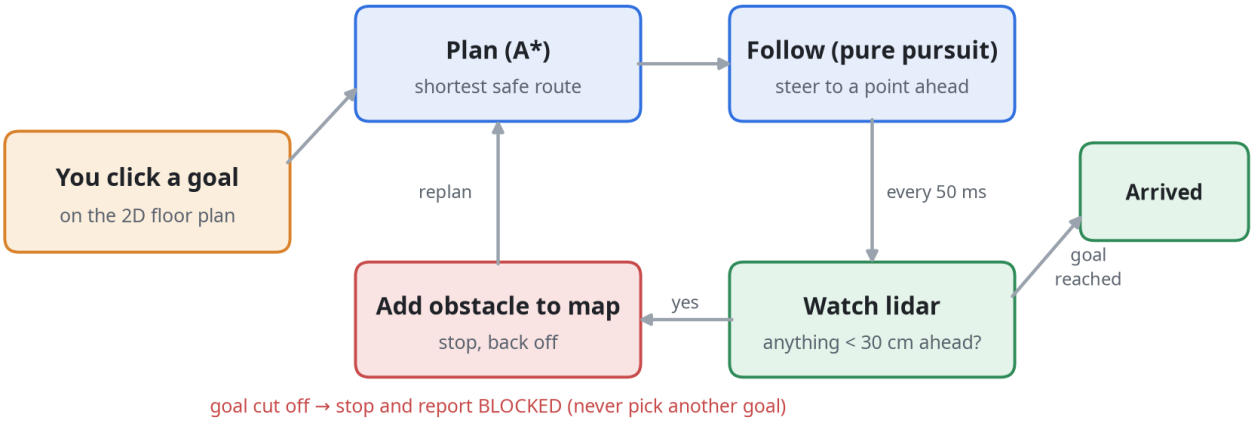
Lesson learned

Our first attempt at run C looked perfect (3.1 cm, 1,500 features in every frame). It was wrong: a texture bug made every triangle show another triangle's colours, like confetti, and the fake detail fooled ORB-SLAM3. We caught it because the simulation had *more* detail than the real video, which is impossible for a copy. When a result looks too good, check it against reality.

One honest caveat: the simulator is now a little *too* easy. Its camera has no motion blur, noise or exposure changes, so its worst frames still have more features than real ones (1,037 vs 583).

9. Making the robot drive itself

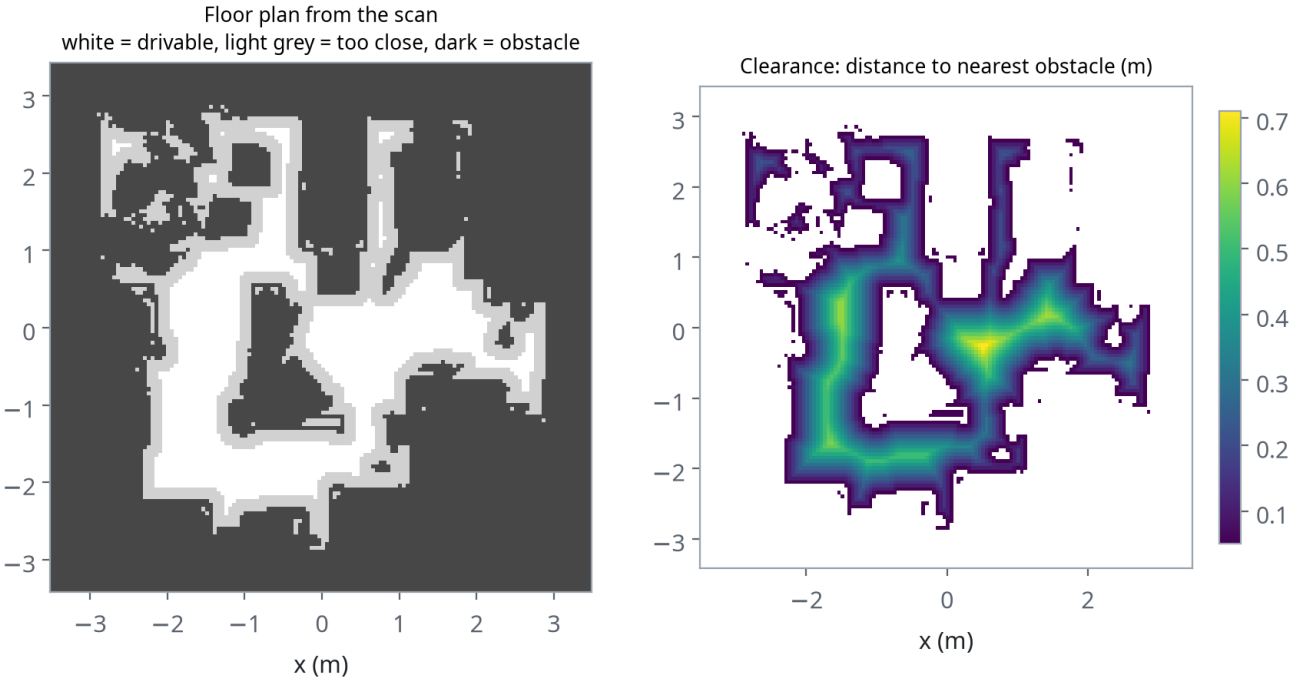
In simple words
You click a point on a floor plan. The robot works out the shortest safe route (A*), follows it smoothly (pure pursuit), and keeps an eye on its lidar. If something unexpected is in the way, it stops, adds it to its map and finds another route, or tells you the goal cannot be reached.



The navigation loop in tools/navigate.py.

9.1 A floor plan from the 3D scan

The 3D model is sliced into 5 cm squares. Floor squares are free; anything between 4 cm and 50 cm high (the robot's height) is an obstacle; floor that was never scanned is treated as unsafe. Then each free square gets its **clearance**: the distance to the nearest obstacle. The robot is about 40 cm wide, so only squares with at least 20 cm clearance are drivable. This trick (called **inflation**) lets the planner treat the robot as a single point.



Left: the floor plan the robot uses. Right: clearance, brighter = more room around you.

9.2 A*: finding the route

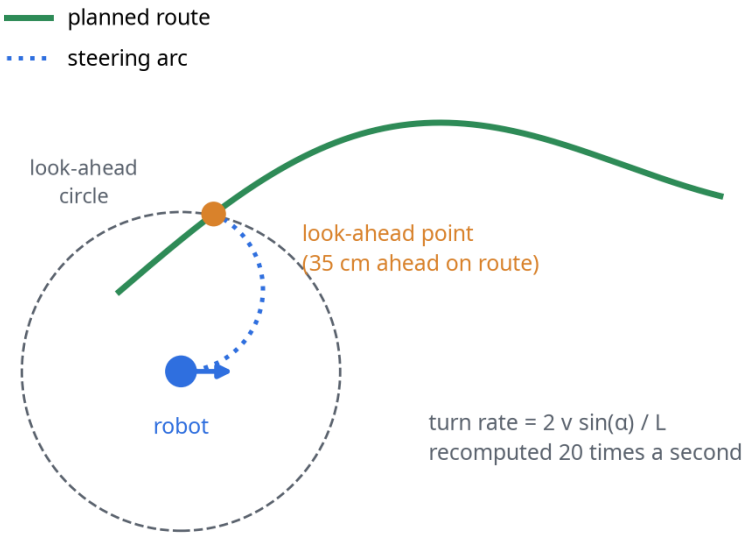
A* searches the grid from the robot to the goal, always expanding the most promising square first (distance travelled so far + straight-line distance still to go). This finds the shortest path quickly. We also make squares near obstacles more “expensive”, so routes stay in the middle of corridors instead of scraping past furniture.



A route planned between two rooms of the apartment.

9.3 Pure pursuit: following the route

Twenty times a second, the robot picks a point on the route 35 cm ahead and steers along the circle arc that reaches it, like a driver looking a little down the road instead of at their feet. For sharp corners it turns on the spot first, and it slows down near the goal.



Pure pursuit: steer along the arc that reaches the look-ahead point.

9.4 Surprise obstacles

If the lidar sees anything closer than 30 cm straight ahead that is not on the map, the robot stops, backs off a little, marks it on its map and plans again to the *same* goal. If the obstacle cuts off the goal completely, it stops and says BLOCKED.

Test	Result
Drive to goals in different rooms (5.7 m and 6.8 m routes)	Arrived within about 10 cm
A box dropped onto the only corridor to the goal	Stopped and reported BLOCKED
A box dropped onto the route in an open area	Replanned 4 times as the lidar saw more of the box; arrived at the real goal

Lesson learned

An early version quietly swapped an unreachable goal for the nearest reachable spot and then reported “Arrived”. A robot must never change its destination without telling you.

9.5 Using it

Run `.venv/bin/python tools/navigate.py`. Gazebo and a map window open. Click on the map to send the robot; the route also appears as a green line in Gazebo. Keys: S or Space = stop, C = forget lidar obstacles, Q = quit. Note: the robot currently knows its exact position from the simulator; using ORB-SLAM3 for this is the next step.

10. Problems we hit and how we fixed them

> **In simple words**
> Twenty problems came up. Most were ordinary (old code, library bugs). The most valuable ones were caught only because a number did not make physical sense.

#	Problem (what we saw)	Cause	Fix
1	Nothing would build	No compiler or CMake installed; needed sudo	Installed build tools from a normal terminal
2	Viewer (Pangolin) failed to compile	Newer compiler needs an explicit header	Added the missing header with a compiler flag
3	ORB-SLAM3 failed against the viewer	Different C++ versions	Compiled both as C++17
4	Error: ++ on a true/false variable	Not allowed in C++17	Changed the variable to a number
5	Our map-export function did not compile	Placed outside ORB-SLAM3's namespace	Moved it inside
6	Dense map came out empty	Bug in an old Open3D class; minimum-weight setting	Switched to Open3D's newer VoxelBlockGrid
7	Plot crashed on colours	Averaged colours slightly above 1.0	Clamped colours to 0 to 1
8	Apartment converter crashed	A pattern did not allow a dot in a name	Fixed the pattern
9	Map step skipped after SLAM	ORB-SLAM3 crashes when its viewer closes	Continue if the trajectory file exists
10	“No floor found”	First frame tilted 45°, wrong idea of down	Average down over the whole walk
11	Apartment 1.4 m underground	Sign error in the levelling formula	Fixed; checked floor, ceiling, camera height
12	Robot skipped half its loop	Follower jumped ahead where the route doubles back	Look at most 0.8 m ahead on the route
13	Robot stuck on open floor	Bump in the scanned floor	Flat physics floor; scan kept for looks
14	Planner found no floor	It used the mesh whose floor we removed	Plan from the visual mesh
15	Big model loads slowly	160 MB file	Wait up to 3 minutes for Gazebo
16	Texturing failed / too slow	Unclean mesh; slow tool at 1M triangles	Wrote a fast per-triangle texture baker
17	Too-good result (confetti texture)	Texture image read upside down by Gazebo	Flipped it; marked the run INVALID and redid it
18	Robot “arrived” at the wrong place	Planner silently changed the goal	Keep the real goal; report BLOCKED
19	Gazebo crashed when navigation started	OpenCV's Qt settings leaked into Gazebo	Removed those settings for Gazebo
20	Our own shell commands died	A kill command matched itself	Used exact process names

Three lessons to keep

- **Sanity-check every number against the real world.** Negative camera height, a room 2.75 m tall, a copy with more detail than the original: these checks caught bugs that every program step had silently accepted.
- **Change one thing per experiment.** Only because run B changed nothing but speed could we rule speed out.
- **Never let a planner change the goal without saying so.**

11. How to run everything

All commands run from ~/github/visual_slam.

```
# 1. Build ORB-SLAM3 (once)
tools/build_orbslam3.sh

# 2. SLAM + dense map on the office video (live viewer)
tools/run_orbslam3.sh

# 3. The apartment: convert the dataset, then SLAM + 2 cm map
.venv/bin/python tools/prepare_bundlefusion.py data/apt0/apt0
MAP_ARGS="--voxel 0.02 --depth-max 4" tools/run_orbslam3.sh \
    data/apt0/apt0 data/apt0/apt0/associations.txt data/apt0/apt0.yaml runs/apt0

# 4. Look at a map in 3D
.venv/bin/python tools/view_map.py --run runs/apt0

# 5. Make the HD digital twin for Gazebo
.venv/bin/python tools/export_gazebo.py --run runs/apt0_1cm --name apt0_hd \
    --align-like apt0 --visual-tris 1000000 --tex-size 8192 --texture per-triangle

# 6. Drive the robot yourself (arrow keys, S = stop)
tools/run_gazebo.sh apt0_hd_robot

# 7. Record a SLAM test drive, run SLAM on it, score it
.venv/bin/python tools/sim_collect.py --out data/sim_run --gui --map apt0_hd \
    --world apt0_hd_robot_tall --cam-z 1.20 --obst-top 1.3
tools/run_orbslam3.sh data/sim_run data/sim_run/associations.txt \
    data/sim_run/camera.yaml runs/sim_run
.venv/bin/python tools/eval_segments.py --run runs/sim_run --gt data/sim_run/groundtruth.txt

# 8. Click-to-go navigation
.venv/bin/python tools/navigate.py
```

Folder / file	What it holds
tools/	All pipeline scripts (build, run, map, export, collect, evaluate, navigate)
gazebo/models/	apt0, apt0_hd (apartment twins), scout_bot, scout_bot_tall (robots)
gazebo/worlds/	Worlds with and without the robot, normal and HD
data/	Downloaded datasets and simulator recordings
runs/	ORB-SLAM3 outputs, dense maps, evaluations
results/REPORT.md	Every result table
docs/	This guide, its figures and the scripts that make them
ROADMAP.md, labs/, vslam/	The from-scratch learning course (Lab 0 is ready)

12. What is next

The project works end to end. These are the most valuable next steps, in order:

1. **Save the work in git** as its own repository.
2. **Navigate using ORB-SLAM3** instead of the simulator's perfect position. This closes the loop from SLAM to navigation, the way a real robot works.
3. **Make the simulated camera realistic**: add motion blur, noise and exposure changes, so the simulator is not easier than reality.
4. **Test loop closure** with routes that revisit places.
5. **Move to ROS 2 + Nav2**, the standard robot navigation stack, using the same twin.
6. **Finish the from-scratch labs** and compare your own SLAM against ORB-SLAM3 in the twin.

Known limits today

- Navigation and test recording use the simulator's exact position.
- The apartment twin is frozen: things that moved during the scan are fixed in place, and some floating scan fragments remain.
- The apartment's reference path is another system's estimate, not true ground truth.
- No loop closure fired on the apartment or in the simulator loops.

Glossary

Word	Meaning
A* (A-star)	A search method that finds the shortest path on a grid quickly
ATE	Absolute Trajectory Error: average distance between the estimated and true paths
Bag of words	Describing an image by which “visual words” (feature types) it contains, for fast place search
Bundle adjustment (BA)	Adjusting all camera poses and 3D points together to reduce reprojection error
Clearance / inflation	Distance to the nearest obstacle; growing obstacles by the robot's radius
Collision mesh	Simplified shape used for physics (what the robot bumps into)
Covisibility graph	Links between keyframes that see the same map points
Depth image	An image where each pixel is a distance instead of a colour
Descriptor	A short binary “fingerprint” of a feature, used for matching
Digital twin	A simulated copy of a real place or object
Drift	Error that slowly builds up over time in SLAM
Feature (ORB)	A distinctive small image spot (corner, edge) that can be found again in other images
Gazebo	Free robot simulator: physics, sensors, rendering
Ground truth	The true answer (here: the exact position from motion capture or the simulator)
gz-transport, topic	Gazebo's messaging; a topic is a named channel such as /scan
Intrinsics	Camera numbers fx, fy, cx, cy that describe how 3D maps to pixels
Keyframe	A frame kept in the SLAM map because it shows something new
Lidar	A sensor that measures distances with laser beams in many directions
Loop closure	Recognising a previously visited place and correcting drift
Marching cubes	Turning a voxel distance field into a triangle mesh
Mesh	A 3D surface made of triangles
Morton (Z-order) curve	An ordering that keeps nearby 3D points near each other in a list
Pose	Position + orientation (6 numbers)
PnP	Perspective-n-Point: computing the camera pose from known 3D points seen in an image
Pure pursuit	Steering towards a point a fixed distance ahead on the route
RANSAC	Fitting a model while ignoring wrong data, by trying many random small samples
Reprojection error	Pixel distance between where a 3D point should appear and where it was seen
RGB-D	Colour (RGB) plus depth (D) camera
SDF	Simulation Description Format: Gazebo's XML files for worlds and models
SLAM	Simultaneous Localization And Mapping
Texture / UV	An image wrapped onto a mesh; UV says which image point goes on which surface point
TSDF	Truncated Signed Distance Function: voxels storing distance to the nearest surface
Umeyama alignment	Best rotation, shift (and scale) to line up two point sets
Voxel	A tiny cube of 3D space, like a 3D pixel